From Sensors to On-Device Inference

An Embedded-to-Edge-AI Reference Architecture

Engineering-to-Research Monograph Series, Volume 4 of 10

Alan Biju Palayil

Independent Researcher · Application Development Analyst, Financial Services · Doctoral Researcher, University of the Cumberlands

ORCID: 0009-0004-8302-5090

Version 1.0 · June 2026

DOI: 10.5281/zenodo.20784402

Companion code: AgriEdge (https://github.com/AlanP13/AgriEdge-AI-Edge); SensoryPi (https://github.com/AlanP13/ECE442-IoT-and-Cyber-Physical-Systems)

Keywords: edge AI, on-device inference, TinyML, embedded systems, cyber-physical systems, sensor fusion, model compression, convolutional neural networks, explainable AI, Jetson Nano, Raspberry Pi

---

Engineering-to-Research Monograph Series · Volume 4 of 10

Executive Summary

Problem. The default machine-learning pattern, namely train a large model in the cloud and send data to it, breaks at the edge, where data originates on devices that are power-, memory-, and bandwidth-constrained, and where decisions often must be made in real time and in the physical world. Moving inference onto those devices is not a deployment detail; it is a co-design problem that spans sensing, model, runtime, and software, each of which constrains the others.

Findings. Edge AI is best treated as a four-layer co-design stack, and its central engineering act is principled reduction: shrinking a model through quantization, pruning, and distillation while preserving both the accuracy and the explainability a non-expert user needs to trust and act on a prediction. Richer, well-chosen sensing can substitute for a bigger model, and the non-expert operator at the edge needs an explanation, not just a label.

Framework. This report contributes a four-layer reference architecture, a stated design principle for principled reduction, and two contrasting case studies that instantiate the architecture: AgriEdge, a plant-disease classifier fusing thermal and visual imaging on an NVIDIA Jetson Nano, and SensoryPi, a Raspberry Pi cyber-physical security system performing on-device facial recognition and motion detection over a lightweight messaging fabric. AgriEdge reached about 94.9% validation accuracy across 38 plant-disease classes with a MobileNet-based model.

Recommendations. Match each layer to the constraints of the one below, spend effort on fused sensing and principled reduction rather than raw model size, and design explainability in at the modeling stage. As edge devices begin to decide and act autonomously, explainability becomes the precondition for governance, which is the bridge to the governance and capstone volumes of the series.

Table of Contents

List of Figures

Abstract

The default mental model for machine learning, train a large model in the cloud and send data to it, breaks at the edge, where data originates on power-, memory-, and bandwidth-constrained devices that must often decide in real time and in the physical world. Moving inference onto those devices is a co-design problem across four layers: sensing, model, runtime and hardware, and software engineering, each of which constrains the others. This report develops a reference architecture for embedded-to-edge AI that makes those cross-layer trade-offs explicit, and grounds it in two systems the author built: AgriEdge, a plant-disease classifier fusing thermal and visual imaging on an NVIDIA Jetson Nano, and SensoryPi, a Raspberry Pi cyber-physical security system performing on-device facial recognition. Against the 2017 to 2026 literature on TinyML, model compression, and explainable edge AI, the report argues that the central engineering act of edge AI is principled reduction: shrinking models while preserving the accuracy and the explainability a non-expert operator needs to trust a prediction. It closes by connecting edge explainability to the governance themes of the series.

---

1. Introduction

A camera in a field, a sensor on a machine, a doorbell at a house: these are where most real-world data is born, and they are exactly the places least suited to running large machine-learning models. The conventional answer, ship the data to the cloud and ship a prediction back, fails on at least one of four counts whenever it matters. It is too slow for real-time action, too bandwidth-hungry for remote or dense deployments, too leaky for privacy-sensitive data, and too expensive at scale. The alternative is to move the inference to the data, which is edge AI.

Edge AI is frequently discussed as if it were merely the same model deployed somewhere smaller. It is not. A constrained device imposes hard limits, namely kilobytes to a few gigabytes of memory, milliwatt to single-digit-watt power budgets, and no elastic compute, and those limits propagate backward into every earlier decision: which sensors can be read and how, which model architectures are even loadable, what latency is achievable, and how the software must be structured to remain maintainable. Edge AI is therefore a co-design problem across four layers: sensing, model, runtime and hardware, and software engineering. Optimizing any one in isolation produces a system that fails in integration, which is the recurring lesson of building real edge devices rather than benchmarking models in notebooks.

The contributions are stated in Section 8 and evaluated in Section 7. In brief: a four-layer reference architecture (Section 4), the principled-reduction design principle (Sections 4 and 5), two contrasting case studies with real results (Section 6), and a bridge from edge explainability to governance (Section 9). Limitations are stated in Section 10 and implementation details in Appendix A.

---

2. Related Work

Three strands of work inform this report, summarized in Table 1.

TinyML and on-device inference. A growing literature establishes the rationale and methods for running models on constrained hardware, cataloguing applications, compression techniques, and open challenges [1]. The benefits, longer battery life, privacy, lower latency, and lower cost, are now well documented, as is the central constraint that inference itself dominates memory and energy on small devices [1], [6].

Efficient model architectures and compression. Compact, mobile-first architectures such as MobileNet made on-device vision practical by trading a small amount of accuracy for large

reductions in computation [2], and compression techniques (quantization, pruning, distillation) shrink models further for microcontroller-class targets [1], [9].

Explainable edge AI for non-experts. A rapidly growing applied literature, especially in plant-disease detection, integrates explanation methods such as Grad-CAM [3] into lightweight CNNs so that a non-expert user sees why a diagnosis was made, not just the label [4], [5], [8], and demonstrates that explainability and edge deployment are compatible goals [5], [7].

Table 1. Comparison of the informing literatures.

Area	Main insight	Limitation
TinyML and on-device inference [1], [6]	Inference can run on constrained devices with real benefits	Inference dominates memory and energy; aggressive optimization required
Efficient architectures and compression [1], [2], [9]	Compact models and reduction fit constrained budgets	Reduction can erode accuracy and explainability if done carelessly
Explainable edge AI [3], [4], [5], [8]	Grad-CAM-class explanations make edge predictions trustworthy	Explanation cost and fidelity under heavy compression are open

The gap. Each strand optimizes one layer: TinyML the runtime, compression the model, explainable-AI work the interpretability. What is missing is an integrated, co-design account that treats sensing, model, reduction, runtime, and software as one stack, and that names where the engineering effort actually pays. This report supplies that account and instantiates it in two built systems.

3. The Case for Inference at the Edge

Four forces justify moving inference onto the device, and the AgriEdge design was framed explicitly around them: reduce latency (act in real time without a network round-trip), reduce bandwidth and central-processing load (do not stream raw imagery to a datacenter), preserve privacy (keep sensitive signals local), and lower operational cost and dependence on connectivity (function where networks are intermittent or absent, such as a farm or a remote installation). The TinyML literature converges on the same quartet of benefits as the standing rationale for the field [1], [6].

The corollary is a constraint, not a free lunch. On resource-constrained devices, inference itself dominates memory and energy [1]. The benefits of edge inference are earned only by paying down that cost through the reductions discussed in Section 5. Edge AI is the discipline of buying latency, privacy, and autonomy with a budget of milliwatts and megabytes.

4. The Embedded-to-Edge-AI Reference Architecture

The architecture composes four layers into one stack, shown in Figure 1, read top to bottom, with each layer constrained by the one below.

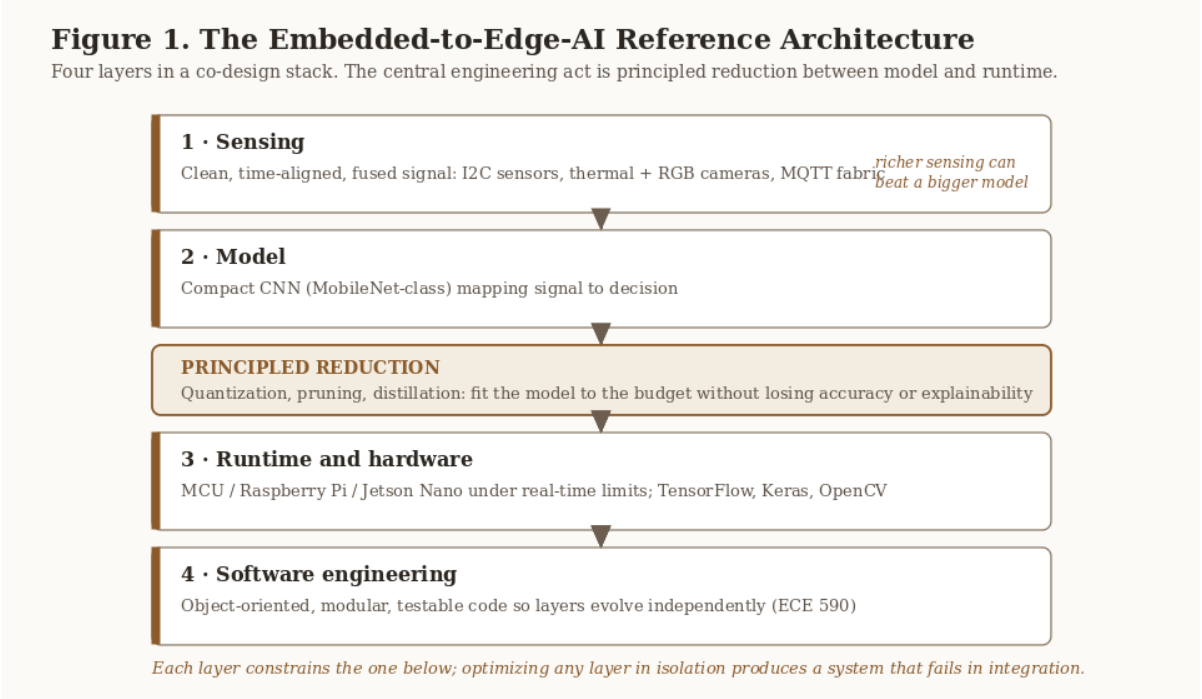

Figure 1. The Embedded-to-Edge-AI Reference Architecture. Four layers in a co-design stack, with principled reduction as the central act between the model and the runtime.

Layer 1, sensing. Clean, time-aligned, fused signal. The author's embedded coursework (ECE 441/442) worked directly with the primitives: reading sensors over I2C, interfacing environmental sensors, and handling serial communication. AgriEdge fuses two visual modalities, a thermographic camera and an RGB camera, plus environmental context from a humidity-temperature sensor. Fusion is deliberate: thermal imaging surfaces physiological stress invisible in RGB, while RGB carries the texture and color cues a disease classifier needs.

Layer 2, model. A compact convolutional neural network maps the signal to a decision. AgriEdge uses a MobileNet-based model [2]; SensoryPi uses on-device facial recognition.

Layer 3, runtime and hardware. The reduced model runs on a specific device tier, from microcontrollers through single-board computers (Raspberry Pi) to edge accelerators (Jetson Nano), under real-time limits, using a portable stack (TensorFlow, Keras, OpenCV).

Layer 4, software engineering. Object-oriented, modular, testable code (the discipline of ECE 590) so the other three layers can evolve independently.

Between the model and the runtime sits the central act of the whole architecture, stated here as a design principle.

Design Principle 1 (Principled Reduction). The core engineering act of edge AI is reducing a model to fit a constrained budget while preserving both the accuracy and the explainability a non-expert user needs to trust and act on its decisions. Reduction that trades away trust is not a saving.

---

5. Principled Reduction

The central engineering act of edge AI is shrinking a model without losing what makes it useful. The literature identifies three complementary techniques [1], [9]: quantization (representing weights and

activations in lower precision, cutting memory and energy with minimal accuracy loss), pruning (removing weights or structures that contribute little), and knowledge distillation (training a small student network to mimic a large teacher). Compact architectures such as MobileNet are themselves a form of designed-in reduction, achieving mobile-grade efficiency by construction [2]. Combined, these techniques can compress models substantially while preserving acceptable accuracy [1].

The word that matters is principled. Reduction is acceptable only insofar as it preserves the properties the application depends on: for AgriEdge, classification accuracy across disease classes and, as Section 9 argues, the explainability of the prediction. This is the content of Design Principle 1.

Key Takeaway. At the edge, better sensing can substitute for a bigger model, and reduction is judged not by compression ratio alone but by whether accuracy and explainability survive it.

---

6. Case Studies

6.1 AgriEdge: plant-disease detection on the Jetson Nano (ECE 501)

System. AgriEdge detects and classifies plant disease in the field. It fuses a thermographic camera (for leaf temperature and vapor-pressure-deficit calculation), an RGB USB camera (for leaf imagery), and an AHT10 temperature-humidity sensor, and runs a MobileNet-based convolutional neural network on an NVIDIA Jetson Nano, with OpenCV extracting frames from the camera feed for real-time inference (Figure 2).

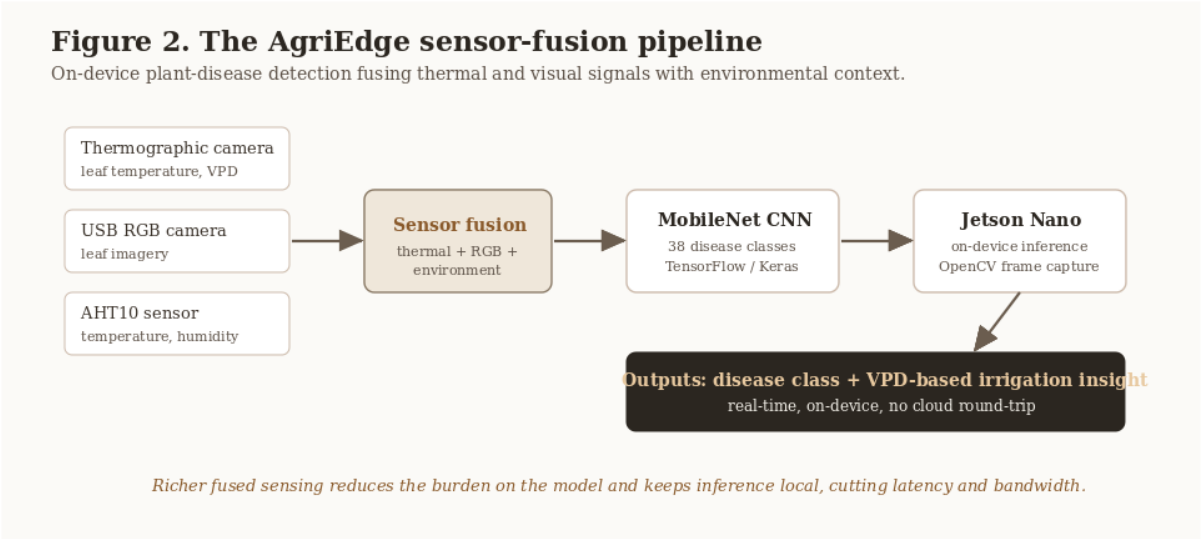

Figure 2. The AgriEdge Sensor-Fusion Pipeline. Thermal, RGB, and environmental inputs are fused, classified by a MobileNet CNN on the Jetson Nano, and turned into a disease class plus a vapor-pressure-deficit irrigation insight, entirely on-device.

Method. The model uses transfer learning from a MobileNet base, with data generators for training and validation, trained for five epochs over a 38-class plant-disease image dataset, and saved for on-device prediction. Training and validation accuracy and loss were tracked per epoch.

Results. The model reached a training accuracy of about 95.4% and a validation accuracy of about 94.9% by the fifth epoch, as shown in Table 2 and Figure 3. The close tracking of training and

validation accuracy across epochs indicates the transfer-learned model generalized well rather than overfitting, which is the desired behavior for a compact model intended to run on constrained hardware.

Table 2. AgriEdge model accuracy by epoch (MobileNet-based CNN, 38 classes).

Epoch	Training accuracy	Validation accuracy
1	0.877	0.910
2	0.931	0.934
3	0.942	0.935
4	0.946	0.944
5	0.954	0.949

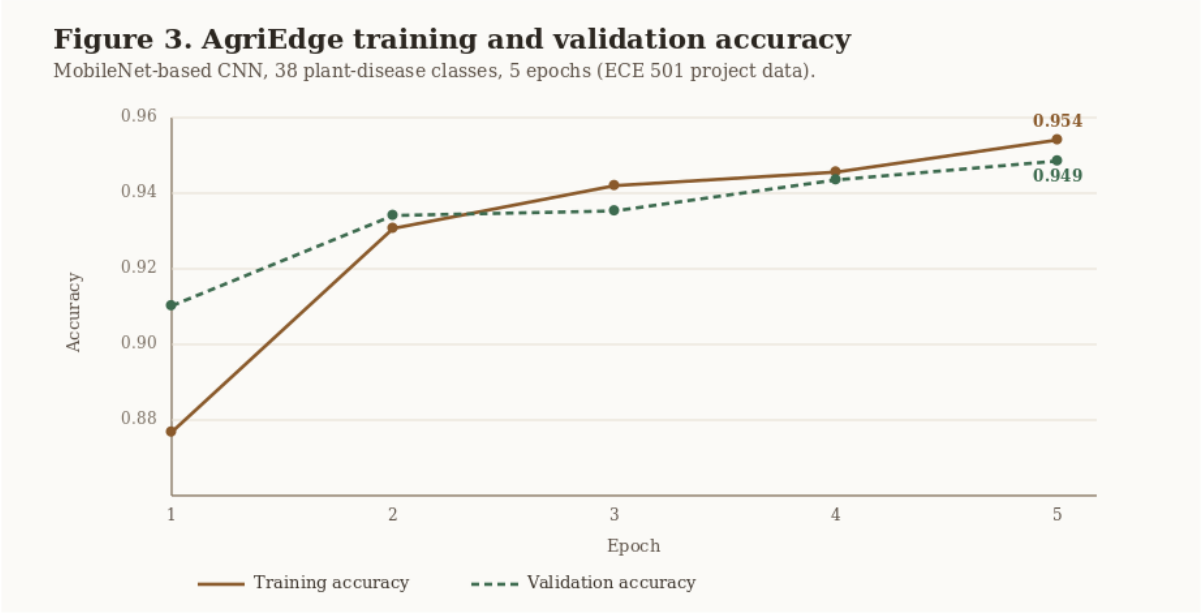

Figure 3. AgriEdge Training and Validation Accuracy. Training and validation accuracy converge over five epochs to about 0.95, the signature of a compact transfer-learned model that generalizes.

6.2 SensoryPi: cyber-physical security on the Raspberry Pi (ECE 442)

System. SensoryPi is a smart security alarm that recognizes and reacts locally. It integrates motion detection and on-device facial recognition on a Raspberry Pi using OpenCV, coordinating its components over MQTT, a lightweight publish-and-subscribe protocol designed for constrained, intermittently connected devices.

What it demonstrates. SensoryPi is the same four-layer architecture in a different domain and a different device tier: sensing (motion and camera), a model (facial recognition), a runtime (Raspberry Pi, OpenCV), and a coordinating fabric (MQTT). It shows that the architecture generalizes beyond agriculture, and it is a concrete instance of pushing both sensing and inference to the device while relying on a thin messaging layer for coordination. (Securing that fabric is the subject of Volume 1.)

What the two studies share. Different domains (agriculture and security), different device tiers (Jetson and Raspberry Pi), and different sensing (fused multi-modal and single-camera), but one four-layer architecture. That shared structure is the point of having a reference architecture at all.

7. Edge Deployment Evaluation

The table compares three ways to deploy an intelligent system that consumes edge-originated data, scored on the criteria that matter for real deployments.

Table 3. Deployment approaches for edge-originated intelligence.

Approach	Latency	Bandwidth use	Privacy	Works offline	Fits device budget	Explainable to user
Cloud inference (send data up)	High	High	Weak	No	N/A	Rarely
Naive edge port (full model on device)	Medium	Low	Strong	Yes	Often no	Rarely
Principled-reduction edge (this report)	Low	Low	Strong	Yes	Yes	Yes, if designed in

Two conclusions follow. First, cloud inference fails the latency, bandwidth, privacy, and offline tests that motivate edge deployment in the first place. Second, naively porting a full model to the device usually fails the budget test, which is why principled reduction (Section 5) is the enabling move rather than an optimization afterthought. The proposed approach provides the best overall balance across the six criteria considered, and the last column, explainability, is satisfied only when it is designed in at the modeling stage (Section 9). The evaluation is structured rather than a single benchmark; a measured latency-and-accuracy comparison across device tiers is named as future work and as a limitation.

8. Synthesized Contributions

This report synthesizes prior work and the author's built systems into four practitioner-oriented contributions, stated here so the reader can locate them.

1. **A generalized four-layer embedded-to-edge-AI reference architecture** (Figure 1) that synthesizes sensing, modeling, runtime constraints, and software engineering into one co-design stack.
2. **The principled-reduction design principle** (Design Principle 1), which holds that reduction must preserve accuracy and explainability, not only shrink the model.
3. **Two contrasting case-study instantiations** (AgriEdge and SensoryPi) that show the architecture holds across domains, device tiers, and sensing modalities, with AgriEdge grounded in real measured accuracy.
4. **A bridge from edge explainability to governance**, drawn out in Section 9, connecting this volume to the governance and capstone volumes of the series.

9. From Accuracy to Trust: Explainability and Governance

An edge prediction is only useful if its user acts on it, and action requires trust. This is acute precisely where edge AI is most valuable, because the non-expert operator at the edge, for example someone acting on an AgriEdge diagnosis or a SensoryPi alert, is typically not a machine-learning expert, and a bare class label gives them no basis to trust or contest it.

The plant-disease literature has moved decisively toward addressing this, integrating explanation methods such as Grad-CAM [3] to produce visual heatmaps showing which leaf regions drove a diagnosis, so the prediction becomes inspectable rather than oracular [4], [5], [8]. Lightweight, interpretable edge classifiers demonstrate that explainability and edge deployment are compatible, not competing, goals [5]. This reframes the principled-reduction thesis of Section 5: the property to preserve under compression is not accuracy alone but accuracy plus the ability to explain.

Practitioner Guidance. Design the explanation in with the model, not after it. On the edge, a heatmap or attribution that a non-expert can read is part of the deliverable, because once a constrained device decides and acts autonomously, the question becomes who is accountable and under what policy. That is the bridge to Volume 6 (risk governance) and the series capstone (Volume 10) on explainable-AI governance.

---

10. Limitations

Four limitations bound this report. First, the AgriEdge results are training and validation accuracy over five epochs on a 38-class image dataset; they are not a field-deployment accuracy under real lighting, occlusion, and sensor-noise conditions, and the team noted that full field integration was constrained by time. Second, the report does not include measured on-device latency, throughput, or energy figures across device tiers; the deployment evaluation in Section 7 is structured rather than benchmarked. Third, SensoryPi is presented at the level of system architecture rather than quantitative performance, because it is a cyber-physical system rather than a benchmarked model. Fourth, the explainability discussion draws on the literature and the architecture's design intent; AgriEdge demonstrates accuracy, while a Grad-CAM-style explanation layer is identified as the next implementation step rather than a completed result.

---

11. Future Work

On-device learning and federation. Moving beyond inference to on-device adaptation and federated learning, where devices improve from local data without centralizing it, would extend the privacy and autonomy benefits while raising new governance questions.

Down to the microcontroller. Pushing AgriEdge-class capability from the Jetson tier toward true TinyML on microcontrollers, via aggressive quantization and distillation, would widen deployment dramatically [1], [9]; the open question is how much explainability survives that degree of reduction.

Measured cross-tier evaluation. Building a benchmark that scores the whole co-design stack, namely latency, energy, accuracy, and explainability across microcontroller, Raspberry Pi, and Jetson tiers, would resolve the second limitation in Section 10.

An explanation layer for AgriEdge. Adding a Grad-CAM-style explanation [3] to the AgriEdge classifier, so a grower sees which leaf regions drove a diagnosis, is the concrete next step that turns the explainability argument of Section 9 into a measured result.

---

12. Conclusion

Edge AI is not a smaller copy of cloud AI; it is a co-design discipline in which sensing, model, reduction, runtime, and software engineering constrain one another, and in which the defining act is principled reduction, shrinking models to a milliwatt-and-megabyte budget without surrendering the accuracy or the explainability that make a prediction trustworthy. AgriEdge, reaching about 94.9% validation accuracy across 38 plant-disease classes on a Jetson Nano, and SensoryPi, performing on-device recognition on a Raspberry Pi, show the same four-layer architecture spanning agriculture and security. And edge AI is where the series turns inward: the moment a constrained device decides and acts in the physical world, explainability ceases to be optional and governance becomes the next problem, which is the problem the capstone takes up.

Acknowledgments

The systems underlying this report originated in coursework at the Illinois Institute of Technology: AI and Edge Computing (ECE 501, the AgriEdge plant-health project, with lab partners and under the instruction of Dr. Jafar Saniie), Internet of Things and Cyber-Physical Systems (ECE 442, the SensoryPi project), and the embedded and object-oriented software coursework (ECE 441, ECE 590). The author acknowledges instructors and project collaborators whose coursework and discussions informed the original systems. All synthesis, analysis, framework development, and writing in this report are the author's own and were written from scratch for publication.

References

[1] S. Heydari and Q. H. Mahmoud, "Tiny Machine Learning and On-Device Inference: A Survey of Applications, Challenges, and Future Directions," Sensors, vol. 25, no. 10, art. 3191, 2025. doi: 10.3390/s25103191.

[2] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017. doi: 10.48550/arXiv.1704.04861.

[3] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," in Proc. IEEE International Conference on Computer Vision (ICCV), 2017, pp. 618-626. doi: 10.1109/ICCV.2017.74.

[4] P. P. Chakrabarti et al., "A lightweight and explainable CNN model for empowering plant disease diagnosis," Scientific Reports, vol. 15, art. 30720, 2025. doi: 10.1038/s41598-025-94083-1.

[5] M. Kowalski et al., "Enhancing agriculture through real-time grape leaf disease classification via an edge device with a lightweight CNN architecture and Grad-CAM," Scientific Reports, vol. 14, art. 16022, 2024. doi: 10.1038/s41598-024-66989-9.

[6] J. Faizi et al., "Deploying TinyML for energy-efficient object detection and communication in low-power edge AI systems," Scientific Reports, vol. 15, art. 44299, 2025. doi: 10.1038/s41598-025-27818-9.

[7] G. Mwitende et al., "AI and IoT-powered edge device optimized for crop pest and disease detection," Scientific Reports, vol. 15, art. 22905, 2025. doi: 10.1038/s41598-025-06452-5.

[8] R. Bansal et al., "LeafAI: Interpretable plant disease detection for edge computing," PLOS ONE, art. e0335956, 2025. doi: 10.1371/journal.pone.0335956.

[9] R. McConville et al., "Optimising TinyML with quantization and distillation of transformer and mamba models for indoor localisation on edge devices," Scientific Reports, vol. 15, art. 10081, 2025. doi: 10.1038/s41598-025-94205-9.

Primary sources, author coursework and built systems, rewritten and synthesized for this report: A. Palayil, "Enhancing Plant Health with AI Using Thermal and Image Processing" (ECE 501, IIT, 2023) and the AgriEdge-AI-Edge repository; the SensoryPi smart security system (ECE 442, IIT, 2022); embedded sensor-integration labs (ECE 441/442); object-oriented machine-learning coursework (ECE 590).

Appendix A. Methodology and Implementation Details

This appendix records the methodological choices behind AgriEdge and SensoryPi so the results are reproducible from the companion repositories.

AgriEdge data and model. The classifier is trained on a 38-class plant-disease image dataset using transfer learning from a MobileNet base. Data generators supply training and validation batches; the model is trained for five epochs, and training and validation accuracy and loss are recorded per epoch and plotted. The trained model is saved and loaded for on-device prediction, and OpenCV extracts frames from the USB camera feed for real-time testing on the Jetson Nano.

AgriEdge sensing. A thermographic camera captures leaf temperature for vapor-pressure-deficit (VPD) calculation, an RGB USB camera supplies leaf imagery for classification, and an AHT10 sensor supplies temperature and humidity. The fused inputs support both disease classification and a VPD-based irrigation insight.

AgriEdge results. Training accuracy rose from about 0.877 to 0.954 and validation accuracy from about 0.910 to 0.949 over five epochs (Table 2, Figure 3). These figures are reproducible from the companion notebook in the AgriEdge repository, where the per-epoch history and saved model are recorded.

SensoryPi. A Raspberry Pi runs motion detection and OpenCV-based facial recognition, with components coordinated over MQTT. The system is implemented in Python with supporting microcontroller code, and is presented here at the level of architecture and method.

Reproducibility. Dependency versions, the dataset split, and the trained-model artifacts are recorded in the companion repositories so the reported accuracy can be regenerated.